COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	04/09/2026	**DATE SUBMITTED:**	08/09/2026
NAME:	Realubit, Seth Andrew I.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 7.1 Sorting Algorithms Pt1

1. PROCEDURE OUTPUT

What procedures did you perform?

main.cpp | sortingAlgo.h

HOA7p1 > Procedures > main.cpp > main()

```cpp
#include <iostream>
#include <cstdlib>
#include <time.h>
#include "sortingAlgo.h"

const int maxSize = 100;

void displayArray(int arr[]);

int main(){

    //create an array with unsorted list with 100 maximum size
    int unsortedArr[maxSize];
    // use time (0) as seed for random values
    srand(time(0));
    //populate the array with random values
    for(int i = 0; i < maxSize; i++){
        unsortedArr[i] = rand()% 100; // random values between 0 and 99
    }
    //display the randomly displayed array
    std::cout << "Unsorted Array: \n" << std::endl;
    displayArray(unsortedArr);
```

```cpp
void displayArray(int arr[]) {

    for (int i = 0; i < maxSize; i++) {
    std::cout << arr[i] << " ";
}
    std::cout << std::endl;
}
```

```
Unsorted Array:

30 77 76 62 42 40 97 8 33 41 47 67 95 25 84 68 34 82 48 29 51 22 5 61 73 54 99 39 33 82 45 26 52 4 70 62 46 3 20 24 96 22 59
12 5 93 15 54 58 23 63 61 86 57 41 12 10 50 18 73 50 95 67 36 72 45 88 36 78 81 0 0 22 51 37 89 31 74 74 34 59 71 10 12 72
4 23 51 32 27 54 80 44 3 33 61 93 13 30 77
```

Code + Console Screenshot

Observations:

Worst-Case Time Complexity: O(1)

Best-Case Time Complexity: O(1)

Table 7-1. Array of Values for Sort Algorithm Testing

```cpp
template <typename T>
void bubbleSort(T arr[], size_t arrSize) {
    //step 1: for i = 0 to N-1 repeat inner loop
    for (int i = 0; i < arrSize; i++) {
        //step 2: inner loop that for K = i+1 to N-1 repeat
        for (int j = i + 1; j < arrSize; j++) {
            //step 3: swap if the arr[j] is greater than arr[i]
            if (arr[i] > arr[j]) {
                //swap arr[j] and arr[i]
                std::swap(arr[i], arr[j]);
            }
        }
    }
}
```

```cpp
std::cout << "\nBubble Sort: \n" << std::endl;
bubbleSort(unsortedArr, maxSize);
displayArray(unsortedArr);
```

Code + Console Screenshot

```cpp
template <typename T>
//array, current position, size of array
int routineSmallest(T arr[], int K, const int arrSize) {
    int position, j;
    T smallestElement = arr[K];

    for (int j = K + 1; j < arrSize; j++) {
        if (arr[j] < smallestElement) {
            smallestElement = arr[j];
            position = j;
        }
    }
    return position;
}

template <typename T>
void selectionSort(T arr[], size_t arrSize) {
    int POS, temp, pass;
    for (int i = 0; i < arrSize; i++) {
        POS = routineSmallest(arr, i, arrSize);
        //swapping
        temp = arr[i];
        arr[i] = arr[POS];
        arr [POS] = temp;

        //for verification and counting the number of passes that was done
        pass++;
        }
    }
```

Observations:

Worst-Case Time Complexity: $O(n^2)$

Best-Case Time Complexity: $O(n^2)$

Table 7-2. Bubble Sort Technique

```cpp
template <typename T>
//array, current position, size of array
int routineSmallest(T arr[], int K, const int arrSize) {
    int position, j;
    T smallestElement = arr[K];

    for (int j = K + 1; j < arrSize; j++) {
        if (arr[j] < smallestElement) {
            smallestElement = arr[j];
            position = j;
        }
    }
    return position;
}

template <typename T>
void selectionSort(T arr[], size_t arrSize) {
    int POS, temp, pass;
    for (int i = 0; i < arrSize; i++) {
        POS = routineSmallest(arr, i, arrSize);
        //swapping
        temp = arr[i];
        arr[i] = arr[POS];
        arr [POS] = temp;

        //for verification and counting the number of passes that was done
        pass++;
        }
    }
```

```
Selection Sort:
0 0 1 1 2 2 3 4 5 6 6 9 12 12 13 16 16 19 19 22 23 23 23 27 29 30 33 36 37 39 39 40 40 41 41 43 44 45 45 46 47 47 47 47 49 49 50 76 52 52 5
2 54 54 54 54 56 57 57 57 58 59 59 61 61 63 64 65 66 67 68 68 68 68 69 70 70 73 73 97 74 74 74 75 77 81 82 82 82 84 85 86 88 89 89 89 99 98
 95 98 98
```

```
std::cout << "\nSelection Sort: \n" << std::endl;
selectionSort(unsortedArr, maxSize);
displayArray(unsortedArr);
```

Code + Console Screenshot

Observations:

Worst-Case Time Complexity: $O(n^2)$

Best-Case Time Complexity: $O(n^2)$

Table 7-3. Selection Sort Algorithm

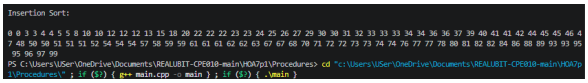
```
template <typename T>
void insertionSort(T arr[], size_t arrSize) {
    for (size_t K = 1; K < arrSize; K++) {
        T temp = arr[K];
        int J = K - 1;

        while (J >= 0 && temp <= arr[J]) {
            arr[J + 1] = arr[J];
            J--;
        }
        arr[J + 1] = temp;
    }

}
```

```
std::cout << "\nInsertion Sort: \n" << std::endl;
insertionSort(unsortedArr, maxSize);
displayArray(unsortedArr);
```

Code + Console Screenshot



Observations:

Worst-Case Time Complexity: $O(n^2)$

Best-Case Time Complexity: $O(n)$

Table 7-4. Insertion Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

```
START
    INITIALIZE candidate names, and array for storing values
    INTIALIZE loop for generating 100 random values with a limit of only 1 to 5 numbers
    DISPLAY unsorted array
    DISPLAY sorted array with insertion sorting implementation from sortingAlgo.h
    DISPLAY sorted array with bubble sorting implementation from sortingAlgo.h
    DISPLAY sorted array with selection sorting implementation from sortingAlgo.h
    IMPLEMENT for loop to increment the values for each vote to the candidate
    DISPLAY the total tallies of each candidate
    DISPLAY the winner of the highest count of tally
END
```

Figure 1. Snippet of the pseudocode

main.cpp | sortingAlgo.h

HOA7p1 > Supplementary > sortingAlgo.h > bubbleSort<T>(T [], size_t)

```cpp
#include <cstdlib>
#include <cstddef>
#include <iostream>
#ifndef SORTING_H
#define SORTING_H

template <typename T>
int bubbleSort(T arr[], size_t arrSize) {
    int passCount = 0;
    //step 1: for i = 0 to N-1 repeat inner loop
    for (int i = 0; i < arrSize; i++) {
        passCount = passCount + 1;
        //step 2: inner loop that for K = i+1 to N-1 repeat
        for (int j = i + 1; j < arrSize; j++) {
            //step 3: swap if the arr[j] is greater than arr[i]
            if (arr[i] > arr[j]) {
                //swap arr[j] and arr[i]
                std::swap(arr[i], arr[j]);
            }
        }
    }
    return passCount;
}

template <typename T>
//array, current position, size of array
int routineSmallest(T arr[], int K, const int arrSize) {
    int position, j;
    T smallestElement = arr[K];
    position = K;

    for (int j = K + 1; j < arrSize; j++) {
        if (arr[j] < smallestElement) {
            smallestElement = arr[j];
            position = j;
        }
    }
    return position;
}

```

Figure 2. Code Snippet of sortingAlgo.h

```cpp
template <typename T>
int selectionSort(T arr[], size_t arrSize) {
    int POS, temp, pass = 0;
    for (int i = 0; i < arrSize; i++) {
        POS = routineSmallest(arr, i, arrSize);
        //swapping
        temp = arr[i];
        arr[i] = arr[POS];
        arr [POS] = temp;

        //for verification and counting the number of passes that was done
        pass++;
        }
    return pass;
}


template <typename T>
int insertionSort(T arr[], size_t arrSize) {
    int passCount = 0;
    for (size_t K = 1; K < arrSize; K++) {
        T temp = arr[K];
        int J = K - 1;

        while (J >= 0 && temp <= arr[J]) {
            arr[J + 1] = arr[J];
            J--;
        }
        arr[J + 1] = temp;
        passCount++;
    }
    return passCount;
}

#endif //SORTING_H
```

Figure 3. Code Snippet of sortingAlgo.h

main.cpp sortingAlgo.h

HOA7p1 > Supplementary > main.cpp > main()

```cpp
#include <iostream>
#include <cstdlib>
#include <time.h>
#include <string>
#include "sortingAlgo.h"

const int maxSize = 100;

void displayArray(int arr[]);

int main(){

    int tallies[6] = {0}; // Array to store tallies for values 1-5

    std::string candidates[6] = {
        "",
        "Bo Dalton Capistrano",
        "Cornelius Raymon Agustin",
        "Deja Jayla Bañaga",
        "Lalla Brielle Yabut",
        "Franklin Relano Castro"
    };

    int unsortedArr[maxSize];
    srand(time(0));
    for(int i = 0; i < maxSize; i++){
        unsortedArr[i] = (rand()% 5)+1;
    }

    std::cout << "Unsorted Array: \n" << std::endl;
    displayArray(unsortedArr);


    int insertionArr[maxSize];
    for (int i = 0; i < maxSize; i++) {
        insertionArr[i] = unsortedArr[i];
```

Figure 4. Code Snippet of main.cpp

```cpp
    std::cout << "\nInsertion Sort: \n" << std::endl;
    insertionSort(unsortedArr, maxSize);
    displayArray(unsortedArr);

    int bubbleArr[maxSize];
    for (int i = 0; i < maxSize; i++) {
        bubbleArr[i] = unsortedArr[i];
    }
    std::cout << "\nBubble Sort: \n" << std::endl;
    bubbleSort(unsortedArr, maxSize);
    displayArray(unsortedArr);

    int selectionArr[maxSize];
    for (int i = 0; i < maxSize; i++) {
        selectionArr[i] = unsortedArr[i];
    }
    std::cout << "\nSelection Sort: \n" << std::endl;
    selectionSort(unsortedArr, maxSize);
    displayArray(unsortedArr);

   for (int i = 0; i < maxSize; i++) {
        tallies[unsortedArr[i]]++; // Increment the tally for the corresponding value
    }

    std::cout << "\nTallies: \n";
    for (int i = 1; i <= 5; i++) {
        std::cout << "Candidate: " << candidates[i] << ": " << tallies[i] << std::endl;
    }

    int Winner = 0;
    int maxTally = 0;
    for (int i = 1; i <= 5; i++) {
        if (tallies[i] > maxTally) {
            maxTally = tallies[i];
            Winner = i;
        }
    }

    std::cout << "Winner: " << candidates[Winner] << std::endl;

}
```

Figure 5. Code Snippet of main.cpp

```
79
80  void displayArray(int arr[]) {
81
82      for (int i = 0; i < maxSize; i++) {
83      std::cout << arr[i] << " ";
84  }
85      std::cout << std::endl;
86  }
```

Figure 6. Code Snippet of main.cpp

```
Unsorted Array:

4 4 2 4 5 3 4 5 4 2 1 3 4 2 3 3 4 5 3 5 5 5 4 2 1 2 5 3 4 5 2 2 1 3 3 3 4 4 1 5 5 3 5 5 3 1 2
1 1 5 4 1 2 2 2 3 4 2 2 2 1 5 2 2 3 1 5 2 1 5 1 5 2 5 4 3 4 5 3 4 2 3 2 1 4 4 2 3 5 3 5 5 4 4
2 3 3 5 1 5

Insertion Sort:

1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 3 3 3
3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5
5 5 5 5 5 5

Bubble Sort:

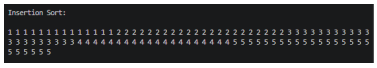
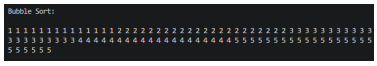
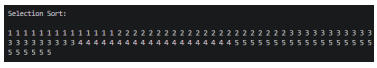
1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 3 3 3
3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5
5 5 5 5 5 5

Selection Sort:

1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 3 3 3
3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5
5 5 5 5 5 5

Tallies:
Candidate: Bo Dalton Capistrano: 14
Candidate: Cornelius Raymon Agustin: 22
Candidate: Deja Jayla Ba▒aga: 20
Candidate: Lalla Brielle Yabut: 20
Candidate: Franklin Relano Castro: 24
Winner: Franklin Relano Castro
PS C:\Users\USer\OneDrive\Documents\REALUBIT-CPE010-main\HOA7p1\Supplementary>
```

Figure 7. Snippet of the Output in the terminal

Output Console Showing Sorted Array	Manual Count	Count Result of Algorithm
	Candidate: Bo Dalton Capistrano: 1 = 1 * 14 Candidate: Cornelius Raymon Agustin: 2 = 1 * 22 Candidate: Deja Jayla Bañaga: 3 = 1 * 20 Candidate: Lalla Brielle Yabut: 4 = 1 * 20 Candidate: Franklin Relano Castro: 5 = 1 * 24	Bo Dalton Capistrano: 14 Cornelius Raymon Agustin: 22 Deja Jayla Bañaga: 20 Lalla Brielle Yabut: 20 Franklin Relano Castro: 24
	Candidate: Bo Dalton Capistrano: 1 = 1 * 14 Candidate: Cornelius Raymon Agustin: 2 = 1 * 22 Candidate: Deja Jayla Bañaga: 3 = 1 * 20 Candidate: Lalla Brielle Yabut: 4 = 1 * 20 Candidate: Franklin Relano Castro: 5 = 1 * 24	Bo Dalton Capistrano: 14 Cornelius Raymon Agustin: 22 Deja Jayla Bañaga: 20 Lalla Brielle Yabut: 20 Franklin Relano Castro: 24
	Bo Dalton Capistrano: 1 = 1 * 14 Cornelius Raymon Agustin: 2 = 1 * 22 Deja Jayla Bañaga: 3 = 1 * 20 Lalla Brielle Yabut: 4 = 1 * 20 Franklin Relano Castro: 5 = 1 * 24	Bo Dalton Capistrano: 14 Cornelius Raymon Agustin: 22 Deja Jayla Bañaga: 20 Lalla Brielle Yabut: 20 Franklin Relano Castro: 24

Question: Yes, the vote counting algorithm proved to be effective because insertion sort only moves when a smaller number is found and because of the amount of duplicates it will stop comparing that number to the others, it would stop moving it.

CONCLUSION

For this hands-on activity I gained a deeper understanding of some of the different sorting algorithms such as bubble sort, selection sort, and lastly insertion sort. Bubble sort is more of going over the list each time it needs to sort something. Then for insertion sort, it moves the sorted list one-by-one, stopping the moment it hits an equal or smaller number. Lastly, selection sorts scans for the minimum and swaps it out to the front and repeat for every single position. During the supplementary activity, I chose an insertion sort because with only values as

high as 5 and having a total of 100, it would produce multiple duplicates, and by using insertion sort it will easily be able to sort through the data (votes). Overall, majority of the things I did was copy one of the sorting algorithms from the procedure and add a compare their tallies to provide the winner (highest tally), thus slightly having improved in coding with the pseudocode as the start.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Claude AI**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explanation of concepts**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Used in trying to deepen my knowledge regarding the different sorting algorithms before I chose which one to choose for the supplementary activity.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.